

Lógica .c

```
#include <stdlib.h>
#include "Jogo.h"
```

```
// Função que cria um baralho com 52 cartas (4 naipes, 13 valores)
```

```
void criarBaralho(jogo *j) {
```

```
    int i = 0;
```

Array começa em 0

```
    for (int naipe = 0; naipe < 4; naipe++) { ← Para cada naipe
```

```
        for (int num = 1; num <= 13; num++) {
```

```
            (*j).baralho[i].num = num;
```

```
            (*j).baralho[i].naipe = naipe;
```

```
            i++;
```

```
        }
```

```
    }
```

```
}
```

Função
mantida
do Goff

← Cria as 13 cartas
Guarda-as na posição i
do baralho

```
// Função que baralha as cartas usando o método Fisher-Yates
```

```
void baralharCartas(jogo *j) { ← começa na última carta
```

```
    for (int i = 51; i > 0; i--) {
```

Função
mantida

Só vai até i-1 → int r = rand() % (i + 1); ← Cria um valor entre 0
daí o +1

← a posição atual

```
        carta temp = (*j).baralho[i];
```

```
        (*j).baralho[i] = (*j).baralho[r];
```

```
        (*j).baralho[r] = temp;
```

```
    }
```

```
}
```

← Troca a carta em i com a carta em r

Percorre a fila de
fãs para a frente
e troca a carta com
outa numa posição
aleatória

Escolhi isto pois
é conhecido e comprovado
que funciona

Função
parc.

alterada

Para as 10
colunas

Corre internamente
tam [i] vezes para
colocar todas as
cartas

```
// Função que distribui as cartas pelas 10 pilhas com quantidades variáveis
```

```
void distribuirCartas(jogo *j) {
```

```
    int tam[10] = {8, 8, 8, 7, 6, 5, 4, 3, 2, 1}; ← Array que define o tamanho das
```

```
    int carta_atual = 51; ← índice da última carta
```

colunas

```
    for (int p = 0; p < 10; p++) {
```

```
        (*j).quantidade_pilhas[p] = tam[p]; ← Atualiza o tamanho da pilha de acordo
```

com o array

```
        for (int c = 0; c < tam[p]; c++) {
```

```
            (*j).pilhas[p][c] = (*j).baralho[carta_atual];
```

```
            carta_atual--;
```

```
        }
```

```
    } (*j).pilhas_concluidas = 0; ← define o nº de pilhas concluídas para 0
```

← vai para a carta anterior

Coloca naquela coluna;
naquela linha

Função
parc.
alterada

```
// Função que invoca as funções necessárias para o começo do jogo
```

```
void comecarJogo(jogo *j) {
```

```
    criarBaralho(j);
```

```
    baralharCartas(j);
```

```
    distribuirCartas(j);
```

```
}
```

Cria a mesa do jogo

```
// Função que verifica se os índices de origem, destino e quantidade são  
válidos
```

recebe jogo

índice origem

índice destino

quantidade de cartas

```
int limitesValidos(jogo *j, int o, int d, int q) {
```

```
    if (o < 0 || o > 9 || d < 0 || d > 9) return 0; ← Se os índices das colunas forem
```

```
    if (q <= 0 || q > (*j).quantidade_pilhas[o]) return 0; ← Se a quantidade for menor
```

```
    return 1;
```

```
}
```

← Se passar nas condições é válido

que o nº de cartas na
pilha ou > 0 é inválido

// Função que verifica se as cartas a mover formam uma sequência válida (mesmo naipe, ordem descendente)

int sequenciaValida(jogo *j, int origem, int qtd) {
 int topo = (*j).quantidade_pilhas[origem] - 1; ← índice da carta no topo
 int base_seq = topo - qtd + 1; ← índice da carta mais baixa a mover
 int erro = 1; ← começa com o erro válido

for(int i = base_seq; i < topo; i++) { ← vai da carta de base até a carta do topo
 carta c1 = (*j).pilhas[origem][i];
 carta c2 = (*j).pilhas[origem][i+1];
 if (erro == 1) { ← verifica se ainda não foi encontrado um erro
 if (c1.naipe != c2.naipe) erro = -2; ← Se os naipes forem ≠ anulada - 2
 else if (c1.num != c2.num + 1) erro = -3; ← Se c1 não for exatamente um valor acima de c2, anula-la - 3
 }
 return erro; ← Devolve 1 se não foi encontrado erro ou devolve o erro

// Função que verifica se o destino aceita a carta a mover (valor imediatamente superior)

int destinoValido(jogo *j, carta movida, int destino) {
 int dest_q = (*j).quantidade_pilhas[destino]; ← quando o n° de cartas na pilha de destino
 if (dest_q == 0) return 1; ← Se a pilha estiver vazia, qualquer carta pode ser movida para lá
 carta topo_dest = (*j).pilhas[destino][dest_q - 1]; ← índice da carta no topo
 if (topo_dest.num != movida.num + 1) return -4;
 return 1; ← Se o índice da carta movida não for imediatamente a baixo, retorna -4
 } ← Se passou as verificações, é válida

// Função que valida todos os aspectos de uma jogada, devolvendo o código de erro adequado (1º erro a ser encontrado)

int validarJogadaTotal(jogo *j, int o, int d, int q) {
 if (limitesValidos(j, o, d, q) == 0) return -1; ← verifica se os valores dados pelo utilizador são válidos
 int seq = sequenciaValida(j, o, q); ← obtém um número que é o erro em seq. válida;
 if (seq != 1) return seq; ← se for ≠ 1, quer dizer que tem um erro devolve o erro dado
 carta c = (*j).pilhas[o][(*j).quantidade_pilhas[o] - q]; ← obtém a carta da base da sequência (aquela que fica em contacto com o topo do destino)
 return destinoValido(j, c, d);
 } ← testa se o destino aceita a carta

// Função que move as cartas da pilha de origem para a pilha de destino

void moverCartas(jogo *j, int orig, int dest, int qtd) {
 int topo_o = (*j).quantidade_pilhas[orig]; ← guarda o n° de cartas na origem
 int base_s = topo_o - qtd; ← calcula o índice da carta na base da sequência (última carta)
 int topo_d = (*j).quantidade_pilhas[dest]; ← n° de cartas na coluna destino
 for (int i = 0; i < qtd; i++) {
 (*j).pilhas[dest][topo_d + i] = (*j).pilhas[orig][base_s + i]; ← copia as cartas de forma ordenada
 }
 (*j).quantidade_pilhas[orig] -= qtd;
 (*j).quantidade_pilhas[dest] += qtd; ← Altera as quantidades
 }

* Exemplo: Se a coluna tem 7 cartas e vamos mover 3, base_s = 4 porque as cartas a mover são os índices [4, 5, 6]

// Função que remove uma sequência completa (Rei ao As do mesmo naipe) se existir no topo da coluna

```
void tentarRemoverSequencia(jogo *j, int col) {
    if ((*j).quantidade_pilhas[col] >= 13) { ← Só vale a pena verificar se tem 13 cartas
        if (sequenciaValida(j, col, 13) == 1) { ← Verifica se formam uma sequência válida
            int t = (*j).quantidade_pilhas[col] - 1; ← índice da carta no topo
            if ((*j).pilhas[col][t].num == 1) { ← verifica se a carta no topo é As
                (*j).quantidade_pilhas[col] -= 13;
                (*j).pilhas_concluidas++;
            }
        }
    }
}
```

sequência válida já garante que tem ordem crescente logo se a primeira for As, está ordenado

Atualiza a quantidade de cartas nas pilhas e as pilhas concluídas

// Função que valida e executa um movimento, devolvendo o resultado da operação

```
int executarMovimento(jogo *j, int o, int d, int q) {
    int validacao = validarJogadaTotal(j, o, d, q); ← guarda o resultado da validação

    if (validacao == 1) { ← Se a jogada for válida
        moverCartas(j, o, d, q); ← altera o estado do jogo
        tentarRemoverSequencia(j, d); ← Verifica se dá para remover a coluna (ou seja, a coluna está completa)
    }

    return validacao;
}
```

devolve o resultado da validação

Códigos:

- 4 → É quando em destinoVálido → Carta no topo da coluna de destino não tem cara imediatamente superior
- 3 → É quando em sequenciaValida → A sequência a mover não está em ordem crescente e com dif. de 1 no valor
- 2 → É quando em sequenciaValida → As cartas têm naipes diferentes
- 1 → É quando em limitesValidos →
 - Coluna de origem ou destino não existe
 - A quantidade de cartas a mover é 0